```
# Import pandas for data handling and analysis
import pandas as pd

# Import numpy for numerical operations
import numpy as np

# Import torch for deep learning tensor operations
import torch

# Import seaborn and matplotlib for visualization
import seaborn as sns
import matplotlib.pyplot as plt

# Import Hugging Face dataset loader
from datasets import load_dataset

# Import BERT tokenizer and model for sequence classification
from transformers import BertTokenizer, BertForSequenceClassification

# Import training utilities from Hugging Face
from transformers import TrainingArguments, Trainer

# Import evaluation metrics from sklearn
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from sklearn.metrics import confusion_matrix, classification_report
```

```
2# Load IMDb dataset from Hugging Face (contains train and test splits)
dataset = load_dataset("imdb")

# Display first 3 records from training dataset
print(dataset["train"][:3])

# Convert training dataset to pandas DataFrame for analysis
df = pd.DataFrame(dataset["train"])

# Print dataset shape (rows, columns)
print("Dataset Shape:", df.shape)

# Display class distribution (0 = negative, 1 = positive)
print(df["label"].value_counts())

# Plot class distribution to check balance
sns.countplot(x="label", data=df)

# Add title to plot
plt.title("Class Distribution (0 = Negative, 1 = Positive)")

# Show plot
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

README.md: 7.81k/? [00:00<00:00, 66.4kB/s]

plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 23.1MB/s]

plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 39.1MB/s]

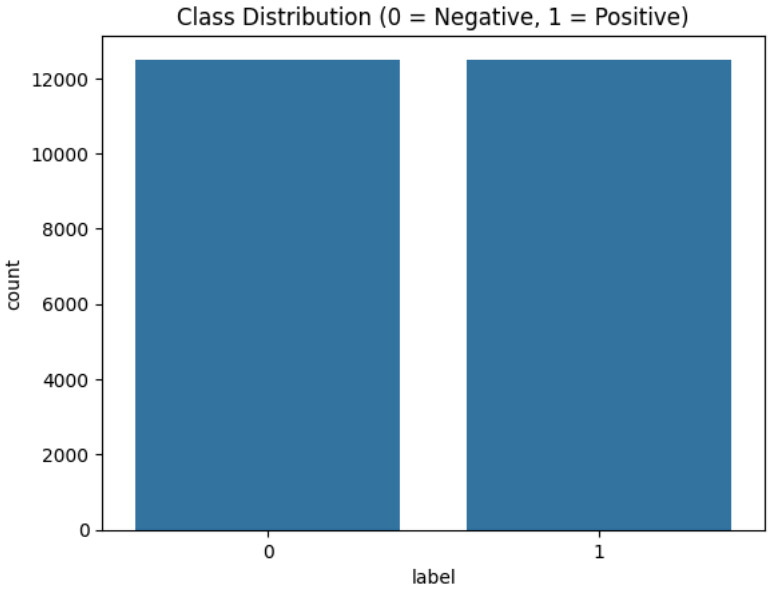
plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:00<00:00, 43.4MB/s]

Generating train split: 100% 25000/25000 [00:00<00:00, 45255.57 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 53261.02 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 64508.30 examples/s]

```
{'text': ['I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was fi
Dataset Shape: (25000, 2)
label
0    12500
1    12500
Name: count, dtype: int64
```




```
# Load pre-trained BERT tokenizer (uncased = lowercase text)
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

# Define tokenization function for dataset mapping
def tokenize_function(example):
    # Convert text into input_ids and attention_mask
    return tokenizer(
        example["text"],            # Input review text
        padding="max_length",       # Pad all sequences to same length
        truncation=True,            # Truncate long reviews
        max_length=256              # Set maximum sequence length
    )

# Apply tokenization to entire dataset (batched for speed)
tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.57kB/s]

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
```

vocab.txt: 100% 232k/232k [00:00<00:00, 4.55MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 3.96MB/s]

Map: 16% 4000/25000 [00:08<00:42, 492.78 examples/s]

```
# Rename label column to 'labels' for Trainer compatibility
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")

# Set dataset format to PyTorch tensors
tokenized_dataset.set_format(
```

```
    type="torch",                               # Convert to torch tensors
    columns=["input_ids", "attention_mask", "labels"]  # Required inputs for BERT
)

# Assign train dataset
train_dataset = tokenized_dataset["train"]

# Assign test dataset
test_dataset = tokenized_dataset["test"]
```

```
# Print number of training samples
print("Train Size:", len(train_dataset))

# Print number of testing samples
print("Test Size:", len(test_dataset))
```

```
# Load BERT model for sequence classification with 2 output labels
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",   # Pre-trained BERT base model
    num_labels=2           # Binary classification (negative, positive)
)
```

```
# Define function to compute evaluation metrics
def compute_metrics(pred):
    # Extract true labels
    labels = pred.label_ids

    # Get predicted class by taking argmax of logits
    preds = pred.predictions.argmax(-1)

    # Compute precision, recall, and F1-score
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average="binary"
    )

    # Compute accuracy
    acc = accuracy_score(labels, preds)

    # Return metrics as dictionary
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }

# Define training arguments
training_args = TrainingArguments(
    output_dir="./results",              # Directory to save model outputs
    learning_rate=2e-5,                  # Learning rate for optimizer
    per_device_train_batch_size=8,       # Batch size for training
    per_device_eval_batch_size=8,        # Batch size for evaluation
    num_train_epochs=2,                  # Number of training epochs
    # evaluation_strategy="epoch",         # Evaluate after each epoch
    save_strategy="epoch",               # Save model after each epoch
    logging_dir="./logs",                # Directory for logs
    # load_best_model_at_end=True          # Load best model based on evaluation
)
```

```
# Initialize Trainer with model, arguments, datasets, tokenizer, and metrics
trainer = Trainer(
    model=model,                         # BERT classification model
    args=training_args,                  # Training configuration
    train_dataset=train_dataset,         # Training data
    eval_dataset=test_dataset,           # Evaluation data
    compute_metrics=compute_metrics      # Evaluation metrics function
)

# Start model training
trainer.train()
```

```
# Predict on test dataset
predictions = trainer.predict(test_dataset)

# Extract true labels
y_true = predictions.label_ids

# Extract predicted labels
y_pred = np.argmax(predictions.predictions, axis=1)
```

```
# Print classification report (precision, recall, F1, accuracy)
print(classification_report(y_true, y_pred))

# Compute confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Create confusion matrix plot
plt.figure(figsize=(6,5))

# Plot heatmap of confusion matrix
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Negative", "Positive"],
            yticklabels=["Negative", "Positive"])

# Label axes
```